ML

06 November 2025 00:33

▶ PCA

Principal Component Analysis (PCA) is a dimensionality reduction technique widely used in machine learning. It transforms high-dimensional data into a lower-dimensional representation while preserving the most important information. PCA is Unsupervised (don't use labels)
It basically,

- Converts a large set of features into a smaller set called **Principal Components** (PCs).
- These PCs keep most of the variance (information) from the original data.

Goal : Maximize Variance

Steps :

1. **Standardize the data** (mean = 0, variance = 1).
2. **Find the covariance matrix** (to see relationships between features).
3. **Calculate eigenvalues & eigenvectors** from the covariance matrix.
4. **Sort eigenvectors** by largest eigenvalues → these represent directions of maximum variance.
5. **Select top k eigenvectors** → these become the new features (PC1, PC2, etc.).

Applications:

- Image compression
- Noise reduction
- Visualization in 2D/3D

Example: If we have 100 features, PCA may reduce them to 10 or 20, but still keep most of the useful information.

▶ Polynomial Regression

Polynomial regression is a statistical method that models the relationship between an independent and a dependent variable as an nth-degree polynomial. It extends linear regression to capture non-linear, curvilinear relationships by fitting a curve to the data, rather than a straight line.

Polynomial Regression is just an **extension of Linear Regression**, used when the data **does not form a straight line**.
In Linear Regression, we try to fit a **straight line (y = m*x + c)** through the data.
But what if the data points form a **curve** instead of a line?
That's when we use **Polynomial Regression** — it helps to fit a **curved line** through the data.

🔹 Example

Let's say you're trying to predict a student's marks based on how many hours they study.

- If the relation is straight (more hours → more marks at a constant rate), linear regression works.
- But in real life, studying **too much** might reduce marks due to stress or tiredness — so the relation looks **curved**.

Polynomial regression can model that curve.

🔹 How it Works

In Linear Regression, the equation is:

$$y = b_0 + b_1 x$$

In **Polynomial Regression**, we add higher powers of x (like x^2, x^3, etc.):

$$y = b_0 + b_1 x + b_2 x^2 + b_3 x^3 + \ldots + b_n x^n$$

So instead of a straight line, this equation gives us a **curve** that better fits the data.

🔹 Example Equation

For a **2nd degree (quadratic)** polynomial:

$$y = b_0 + b_1 x + b_2 x^2$$

For a **3rd degree (cubic)** polynomial:

$$y = b_0 + b_1 x + b_2 x^2 + b_3 x^3$$

Each higher power allows the model to "bend" more and capture complex patterns.

▶ Ridge Regression

Ridge regression, also known as L2 regularization, is a technique used in linear regression to address the problem of multicollinearity among predictor variables. Multicollinearity occurs when independent variables in a regression model are highly correlated, which can lead to unreliable and unstable estimates of regression coefficients.

Ridge Regression Adds a penalty on large coefficients to shrink them → keeps coefficients small → reduces variance → prevents overfitting.

Effect:

- Coefficients shrink **towards zero** but never become exactly zero.
- Helps reduce **overfitting**.

Pros:

- Works well when many features are correlated.
- Reduces variance.

Cons:

- Does not perform feature selection (all features remain in the model).

▶ Lasso Regression

Lasso Regression is a regularization technique used to prevent overfitting. It improves linear regression by adding a penalty term to the standard regression equation. It works by minimizing the (MSE) sum of squared differences between the observed and predicted values by fitting a line to the data. It helps to deal with multicollinearity

Effect:

- Some coefficients become **exactly zero**.
- Performs **feature selection** (automatically removes less important features).

Pros:

- Simple and interpretable.
- Works well when we have many irrelevant features.

Cons:

- If features are highly correlated, Lasso may arbitrarily pick one and drop others.

▶ Logistic Regression

Logistic regression is a statistical method used to predict the probability of a binary outcome (like "yes" or "no," "true" or "false") based on one or more input variables.
It's based on Linear Regression, but instead of a straight line, it uses the **Sigmoid function** to squash output between **0 and 1**.

▶ SVM?

SVM (Support Vector Machine) is a **supervised machine learning algorithm** used for **classification** and sometimes **regression**.
👉 Its main goal is to **find the best boundary (line or surface)** that separates data points of different classes.
Example:
Suppose you have data of **cats** and **dogs** — SVM tries to draw the **best line** (in 2D) or **plane** (in higher dimensions) that separates them clearly.

🔹 1️⃣ The Basic Idea

SVM looks for the **optimal separating line (called a Hyperplane)** that divides different classes with the **maximum margin**.
Let's break those words 👇

🔸 Hyperplane

A **hyperplane** is just a line (in 2D) or a plane (in 3D or higher) that divides the data.

- For 2D data → it's a **line**
- For 3D data → it's a **plane**

Example:
If you have two features (say, weight and height), the SVM will find a **line** that best separates the two classes.

🔸 Margin

Margin = the **distance between the hyperplane and the nearest data points** from each class.
SVM tries to make this margin **as wide as possible**, because:

- **A larger margin** → better separation
- **Smaller margin** → higher chance of misclassification

🔸 Support Vectors

The **closest data points** to the hyperplane (from both classes) are called **Support Vectors**.
They are "supporting" the position of the hyperplane — if you move or remove one, the boundary might change!
So these are the **most important points** in the dataset.

🔹 2️⃣ How SVM Works (Step by Step)

1. The algorithm takes all your data points.
2. It tries to find a **hyperplane** that separates classes (e.g., cat vs dog).
3. It checks which line (or plane) gives the **maximum margin** between classes.
4. That **optimal hyperplane** is the final decision boundary.

🔹 3️⃣ Linearly Separable vs Non-Linearly Separable Data

🟩 **Linearly Separable**
When data can be separated by a straight line.
➡️ Example: Simple case of cats vs dogs clearly divided.
Here, SVM easily finds a **straight line** (hyperplane).